

Group Activity 03

(3인 혹은 4인으로 팀을 구성하여 아래의 문제를 푼다. 팀 구성은 매 시간마다 달라져도 된다.)

팀원1: _____

팀원2: _____

팀원3: _____

팀원4: _____

1. 삽입정렬(insertion sort)에서 데이터를 삽입할 위치를 이진검색(binary search) 알고리즘으로 찾을 수도 있다. 그렇게 한다면 삽입정렬 알고리즘의 최악의 경우 시간복잡도는? 이유는?

2. 합병(merge) 정렬 알고리즘에서 데이터를 절반으로 나누지 않고 항상 1/3과 2/3로 나눈다면 최악의 경우 시간 복잡도는? 이유는?

3. n 개의 정수의 수열 $a_1, a_2, \dots, a_n$ 에 대해서 $i < j$ 이면서 $a_i > a_j$ 일 때 (a_i, a_j) 는 역전된(inverted) 쌍이라고 부른다. 입력에서 역전된 쌍의 개수가 최대 n 개를 넘지 않는다고 가정하면 삽입정렬 알고리즘의 최악의 경우 시간 복잡도는? 이유는?

4. n 개의 정수들 중에서 $n/4$ 번째로 작은 정수를 $O(n)$ 시간에 찾는 알고리즘이 존재한다. 이 알고리즘을 이용하여 항상 $n/4$ 번째로 작은 정수를 피벗(pivot)으로 선택하여 사용한다면 quicksort 알고리즘의 최악의 경우 시간 복잡도는? 이유는?

5. 안정된 정렬(stable sort) 알고리즘이란 동일한 값들은 입력된 순서대로 출력되는 정렬 알고리즘을 말한다. 버블정렬, 선택 정렬, 삽입정렬, 합병정렬, 빠른정렬은 각각 안정된 정렬 알고리즘인가? 만약 불분명하다면 어떻게 해야 안정된 정렬 알고리즘이 되는지 설명하라.

6. 만약 정렬할 데이터들이 모두 동일한 값이라면 다음 중 어떤 알고리즘이 가장 적은 시간이 걸릴까? 이유는?
(1) 삽입정렬 (2) 힙 정렬 (3) 합병정렬 (4) 선택정렬

7. N 개의 값을 가진 이진 max-heap이 있다. 이중 최소값을 찾기 위한 알고리즘을 구상하라. 시간복잡도는? 이유는?

8. 빈(empty) 상태의 이진 최대(max) 힙에 32, 15, 20, 12, 25, 16이 순서대로 삽입되었을 때 만들어지는 힙의 모양을 트리로 그려라.

9. 배열로 표현했을 때 40, 30, 20, 10, 15, 16, 17, 8, 4인 이진 최대 힙이 있다. 여기에 35를 삽입했을 때 만들어지는 힙을 배열로 표현하면?

10. 각각 크기가 N 인 두 개의 max heap이 있다. 두 max heap을 합쳐서 $2N$ 개의 원소를 가진 하나의 max heap을 만들기 위해서 필요한 시간복잡도는?

11. 이진 최소(min) 힙이 있다. 힙에 속한 어떤 한 원소의 값을 원래 값보다 더 작은 값으로 변경하는 연산의 시간 복잡도는? 물론 변경 후에도 힙의 특성을 유지해야 한다.

12. N 개의 ASCII 코드 문자들을 정렬한다. 다음 중 가장 효율적인 알고리즘은? 이유는?

- (1) Quicksort (2) Heap sort (3) Merge sort (4) Counting sort

13. 정렬되지 않은 N 개의 정수에 대해서 최소값을 구할 때 일반적으로 $n - 1$ 번의 비교 연산이 필요하다. 더 적은 횟수의 비교 연산으로 최소값을 구할수 있을까? 왜?

14. 정렬되지 않은 서로 다른 N 개의 정수에 대해서 최소값과 2번째로 작은 값을 구하려 한다. 생각할 수 있는 가장 간단한 방법은 먼저 $n - 1$ 번의 비교 연산을 수행하여 최소값을 찾고, 다시 최소값을 제외한 나머지 $n - 1$ 값들에 대해서 $n - 2$ 번의 비교연산을 해서 최소값을 찾는 것이다. 이 경우 비교 연산의 횟수는 $2n - 3$ 번이 된다. 더 적은 횟수의 비교 연산으로 최소값과 2번째로 작은 값을 구할수 있을까? 어떻게?

15. 오름차순으로 정렬된 N 개의 정수에 대해서 이진검색을 하면 최악의 경우 시간복잡도는 $O(\log n)$ 이다. 최악의 경우 시간복잡도가 이것보다 더 낮은 알고리즘이 존재할 수 있을까? 왜?

16. $m \times n$ 크기의 2차원 배열 A 가 있다. 즉, 행의 개수는 m 이고 열의 개수는 n 이다. 각 행은 오름차순으로 정렬되어 있고, 각 열도 오름차순으로 정렬되어 있다. 예를 들면 다음과 같다.

```
int arr[][] = [ [23, 25, 35, 37, 42],
                [27, 31, 39, 40, 43],
                [38, 39, 43, 51, 52],
                [41, 42, 45, 53, 55],
                [44, 48, 51, 56, 61]];
```

이 배열에 어떤 값 $target$ 이 있는지 검색하는 문제를 생각해보자. 단순히 배열의 모든 값을 검색할 경우 시간복잡도는 (A)가 될 것이다. 각 행이 정렬되어 있다는 것을 이용하여 각 행에 대해서 이진검색을 한다면 시간복잡도는 (B)가 될 것이다. 마찬가지로 각 열이 정렬되어 있다는 것을 이용하여 각 열에 대해서 이진검색을 한다면 시간복잡도는 (C)가 될 것이다. 다음과 같은 분할정복법 알고리즘을 생각할 수 있다. 이 배열을 가로와 세로로 2등분한 배열을 각각 A_{00} , A_{01} , A_{10} , A_{11} 이라고 하자.

$A_{00}=A[0...m/2][0...n/2]$, $A_{01}=A[0...m/2][n/2...n]$, $A_{10}=A[m/2...m][0...n/2]$, $A_{11}=A[m/2...m][n/2...n]$
가운데에 해당하는 값인 $data[m/2][n/2]$ 를 $target$ 과 비교하여 만약 $data[m/2][n/2] > target$ 이면 배열 A_{11} 에는 $target$ 값이 있을 수 없으므로 A_{00} , A_{01} , A_{10} 각각에 대해서 recursion으로 다시 검색하면 된다. 반대로 $data[m/2][n/2] < target$ 이면 A_{01} , A_{10} , A_{11} 각각에 대해서 recursion으로 다시 검색하면 된다. 이 알고

리즘의 시간복잡도는 (D)가 된다. (A), (B), (C), (D) 각각에 들어갈 시간복잡도는? (D)를 계산할 때는 $m = n = 2^k$ 라고 가정해도 된다. 이 문제에 대해서는 위의 4가지 방법보다 더 나은 알고리즘이 존재한다. 이 알고리즘에 대해 알고 있다면 간단히 기술하라.